



Projet de Fin d'Études

Analyse Prédictive sur des Données Réelles

Parcours Data Analyst

Année universitaire 2024–2025

Réalisé par : Aritendry Ramahafeno
En collaboration avec : Bernardin RASOLONJANAHARY
Encadré par : Docteur Dimby

Table des matières

Remerciements	3
Résumé	4
Abstract	5
1 Introduction Générale	6
1.1 Contexte	6
1.2 Problématique	6
1.3 Objectifs	6
1.4 Démarche suivie	6
1.5 Structure du rapport	6
1.6 Exploration des données	7
1.6.1 Chargement du jeu de données	7
1.6.2 Aperçu des données	7
1.6.3 Traitement des valeurs manquantes	7
1.6.4 Détection de doublons	7
1.6.5 Étude des valeurs aberrantes	8
1.6.6 Étude des corrélations	8
1.6.7 Conclusion de l'exploration	8
2 Visualisation des Données	9
2.1 Introduction	9
2.2 Analyse univariée	10
2.3 Analyse bivariée	16
2.4 Conclusion	20
2.5 Présentation du Dataset	20
3 Modélisation	25
3.1 Évaluation des modèles	25
3.2 Résultats obtenus	25
3.3 Courbes d'apprentissage	25
3.4 Analyse des résultats	28
4 Déploiement	29
4.1 Test interactif du modèle	29
4.2 Chargement du modèle	29
4.3 Exemple d'utilisation	29

Annexes	31
Bibliographie	40
5 Conclusion	41

Remerciements

D'abord, on veut vraiment dire merci à Dieu qui nous a accompagnés pendant tout ce projet de fin d'études. Sans Sa grâce, on aurait pas eu la force ni le courage d'aller au bout.

On tient aussi à remercier le Dr Dimby qui nous a supervisés. Ses conseils et sa disponibilité ont été super précieux pour nous, surtout quand on bloquait sur certains points.

Un grand merci aussi à tous les profs du département d'Informatique de l'INSI. Les cours qu'on a eus pendant les deux semestres nous ont vraiment préparés pour ce projet.

Et pour finir, merci du fond du cœur à nos familles. Leur soutien et leurs encouragements ont été super importants pour nous, surtout dans les moments un peu difficiles.

Résumé

Dans ce travail de fin d'études, on s'est penché sur la prédiction des chances de réussite aux concours d'entrée à l'université. On est parti d'un jeu de données avec différentes variables académiques - les scores GRE et TOEFL, la moyenne générale CGPA, et quelques autres critères qui peuvent jouer.

Le boulot a consisté d'abord à nettoyer et préparer ces données, puis à tester plusieurs approches de modélisation. Finalement, on a opté pour une forêt aléatoire (Random Forest) qui semblait donner les meilleurs résultats pour prédire l'admission des candidats.

Les résultats sont plutôt bons, avec une précision qui dépasse 90

Abstract

This final year project tackles the prediction of success rates in university entrance exams. We worked with a dataset containing various academic indicators including GRE and TOEFL scores, CGPA, and other relevant factors that might influence admission chances.

The work involved cleaning and preparing the data, then experimenting with different modeling approaches. We ultimately selected a Random Forest classifier which showed the best performance in predicting candidate admission.

The results are quite encouraging, with accuracy exceeding 90

Chapitre 1

Introduction Générale

1.1 Contexte

L'entrée à l'université représente un enjeu important pour beaucoup d'étudiants, et les concours d'admission sont souvent une étape décisive. Pouvoir prédire les chances de réussite permet non seulement d'aider les candidats à mieux se préparer, mais aussi d'accompagner les établissements dans leur processus de sélection.

1.2 Problématique

Prédire si un candidat va réussir son concours d'entrée est un vrai défi : les données sont nombreuses, variées, et pas toujours très propres. Difficile, dans ces conditions, de bâtir un modèle fiable. Il fallait donc trouver une manière efficace de traiter ces informations pour en tirer des prédictions utiles.

1.3 Objectifs

L'objectif de ce projet était de développer un modèle capable d'estimer les probabilités de réussite au concours, en se basant sur des données académiques comme les scores au GRE ou au TOEFL, la moyenne générale, et d'autres critères pertinents.

1.4 Démarche suivie

Pour y parvenir, on a commencé par rassembler plusieurs jeux de données, puis on en a choisi un — le plus complet et fiable. Ensuite, on a nettoyé et préparé les données, avant de les explorer pour bien comprendre ce qu'elles contenaient. Plusieurs algorithmes de machine learning ont été testés et comparés pour sélectionner celui qui donnait les meilleurs résultats.

1.5 Structure du rapport

Ce mémoire est organisé en quatre parties :

- La première partie présente la collecte et le prétraitement des données.

- La deuxième détaille la modélisation et l'évaluation des différents algorithmes.
- La troisième analyse les résultats et les performances du modèle retenu.
- Enfin, la dernière partie discute des limitations et propose des pistes d'amélioration pour aller plus loin.

1.6 Exploration des données

1.6.1 Chargement du jeu de données

Le jeu de données a été chargé au format .CSV à l'aide de la bibliothèque python pandas :

```
import pandas as pd
df = pd.read_csv("admission_data.csv")
```

La commande `df.info()` permet de voir une partie des colonnes, de leurs types et du nombre de valeurs non nulles. On compte 9 colonnes et 550 lignes.

1.6.2 Aperçu des données

- Le jeu de données contient des variables quantitatives comme par exemple (*GRE Score*, *TOEFL Score*, *CGPA*, *Chance of Admit*) et discrètes (*University Rating*, *SOP*, *LOR*, *Research*).
- Ici *Chance of Admit* est la variable cible. Elle indique la probabilité d'admission.

1.6.3 Traitement des valeurs manquantes

Des valeurs manquantes ont été détectées dans deux colonnes :

- **TOEFL Score**
- **SOP (Statement of Purpose)**

Ces deux colonnes présentaient chacune 37 valeurs manquantes.

La stratégie choisie a été de supprimer toutes les lignes contenant au moins une valeur manquante :

```
df_clean = df.dropna()
```

Puis on a fait une vérification pour s'il n'y a plus de variable manquantes dans le dataset

```
df_clean.isnull().sum()
```

Résultat : toutes les valeurs manquantes ont été supprimées. Le jeu de données final contient 513 lignes.

1.6.4 Détection de doublons

Pour vérifier la présence de doublons, j'ai utilisé la commande suivante :

```
df_clean.duplicated().sum()
```

Résultat : aucune ligne dupliquée n'a été détectée.

1.6.5 Étude des valeurs aberrantes

Des diagrammes en boîte (*boxplots*) ont été tracés pour les variables numériques afin de visualiser les potentiels outliers. Bien que certains points soient situés à l'extérieur du graphe de boîtes à moustache, aucun n'a été jugé significativement aberrant ou incohérent (par exemple, un CGPA supérieur à 10 ou un score GRE supérieur à 340). Aucun traitement particulier n'a été effectué à ce niveau.

1.6.6 Étude des corrélations

Une matrice de corrélation a été générée pour évaluer les relations entre les variables :

```
corr_matrix = df_clean.corr()
```

Les coefficients de corrélation les plus élevés avec la variable cible *Chance of Admit* sont :

- **CGPA** : 0.87
- **GRE Score** : 0.82
- **TOEFL Score** : 0.79
- **Research** : 0.55

On voit que les performances académiques (CGPA, GRE, TOEFL) et l'expérience de recherche sont fortement corrélés aux chances d'admission.

1.6.7 Conclusion de l'exploration

L'analyse exploratoire a permis de :

- Nettoyer le jeu de données (suppression des lignes incomplètes),
- Confirmer l'absence de doublons,
- Identifier les variables les plus corrélées à la cible,
- Préparer les données pour les étapes de visualisation et de modélisation.

Chapitre 2

Visualisation des Données

2.1 Introduction

La visualisation des données constitue une étape cruciale dans l'analyse exploratoire. Elle permet de :

- comprendre la distribution des variables,
- détecter des tendances ou des regroupements,
- observer les relations entre variables explicatives et la variable cible (*Chance of Admit*),
- orienter les choix de modélisation futurs.

Nous distinguons ici deux types d'analyse : univariée (étude d'une seule variable) et bivariée (étude de la relation entre deux variables).

2.2 Analyse univariée

GRE Score

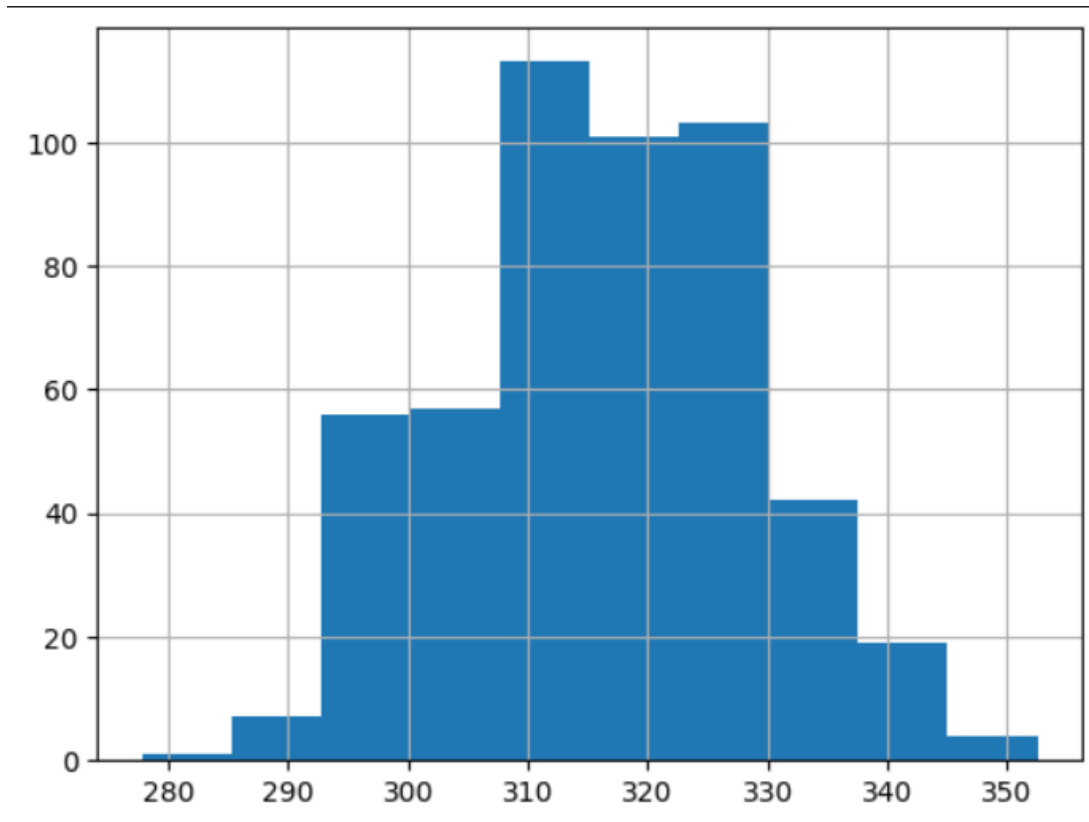


FIGURE 2.1 – Distribution du GRE Score

La majorité des scores GRE se situe entre 300 et 335. Cela montre une répartition relativement homogène avec une concentration autour de scores élevés, ce qui peut refléter un ensemble de candidats compétitifs.

CGPA

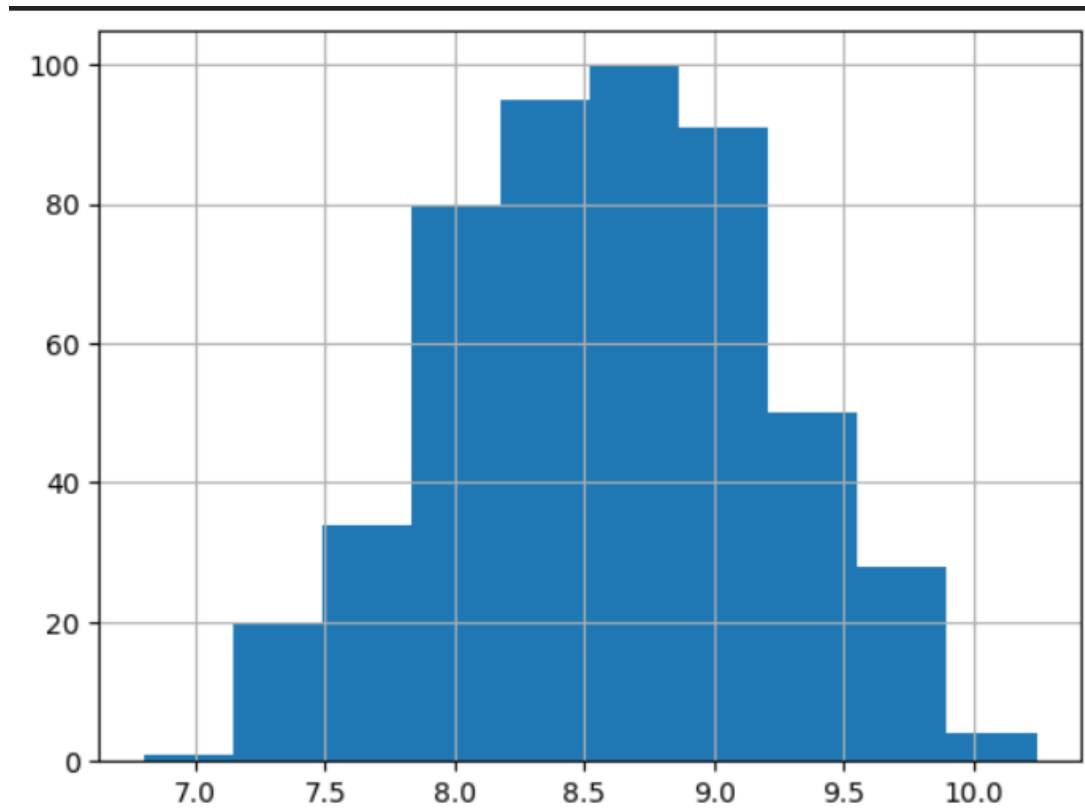


FIGURE 2.2 – Distribution du CGPA

On observe une forte concentration des CGPA entre 8.0 et 9.5. Très peu de candidats ont un CGPA inférieur à 7.5. Entre 9.5 et 10, on note aussi une densité significative, indiquant que de nombreux candidats obtiennent des résultats académiques très solides.

TOEFL Score

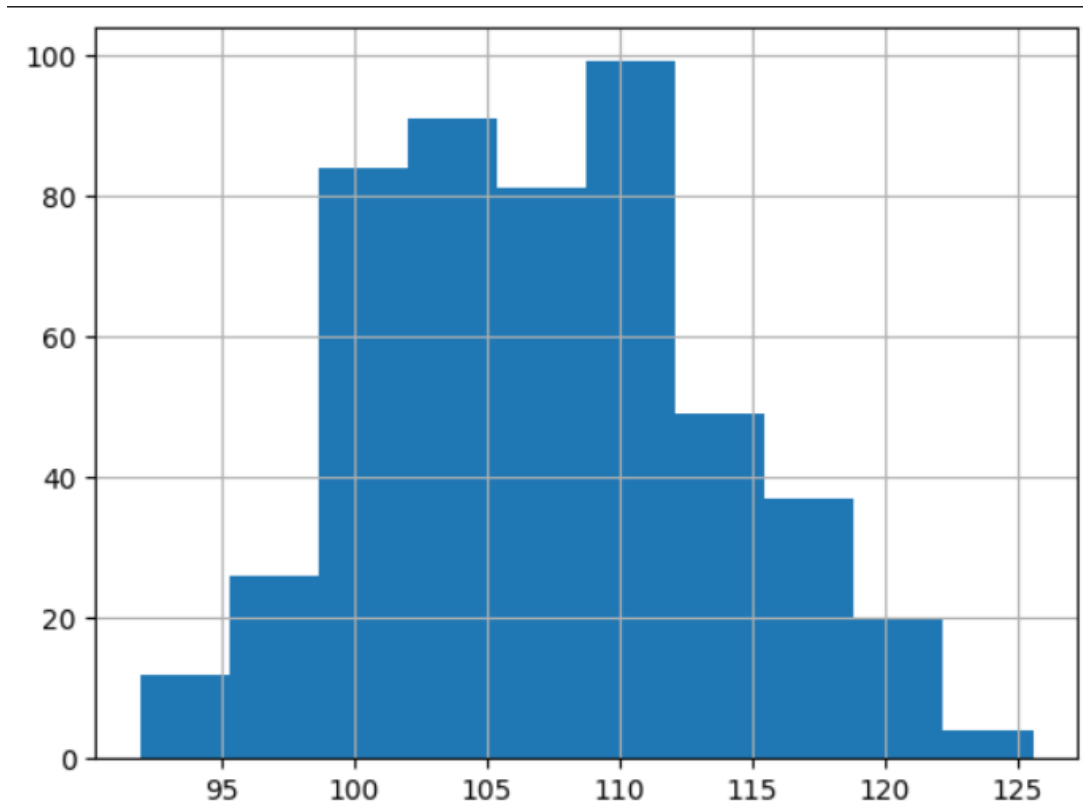


FIGURE 2.3 – Distribution du TOEFL score

Les scores TOEFL se concentrent principalement entre 100 et 110, ce qui est typique d'étudiants internationaux préparés. Quelques candidats atteignent même des scores supérieurs à 115.

University Rating

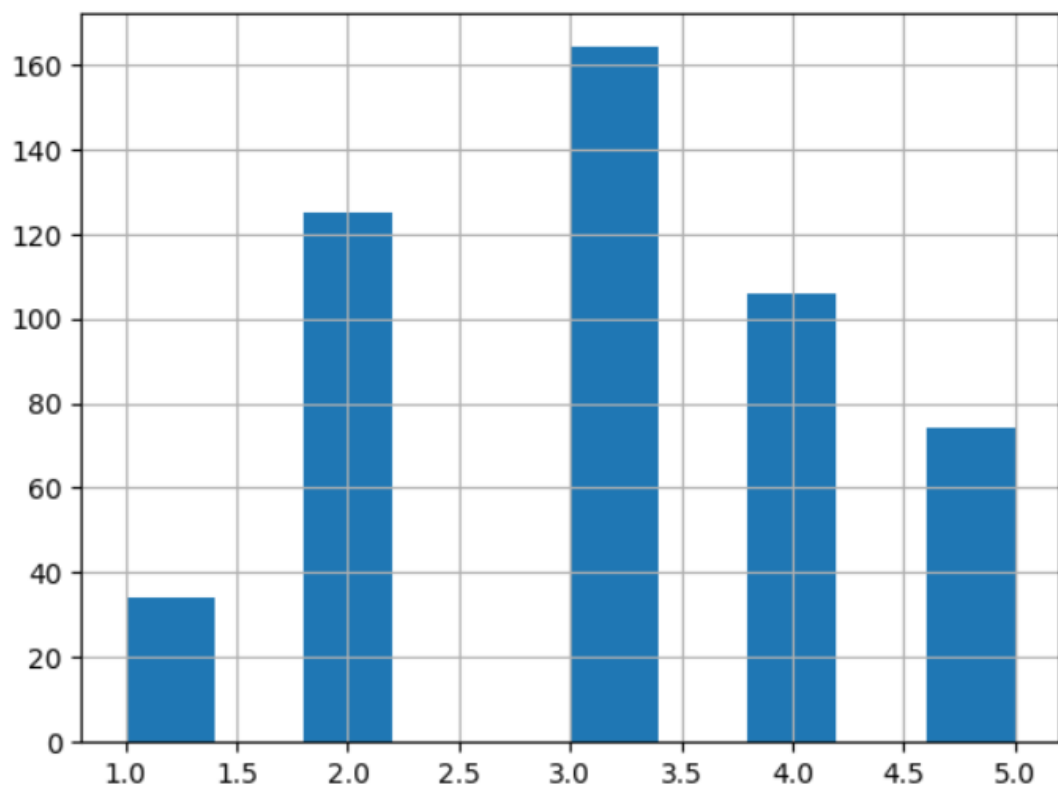


FIGURE 2.4 – Distribution de l'évaluation des universités

Les notes d'université sont principalement réparties entre 2 et 4. Très peu de candidats proviennent d'universités notées 1 ou 5, ce qui suggère une tendance vers des établissements modérément bien classés.

Research

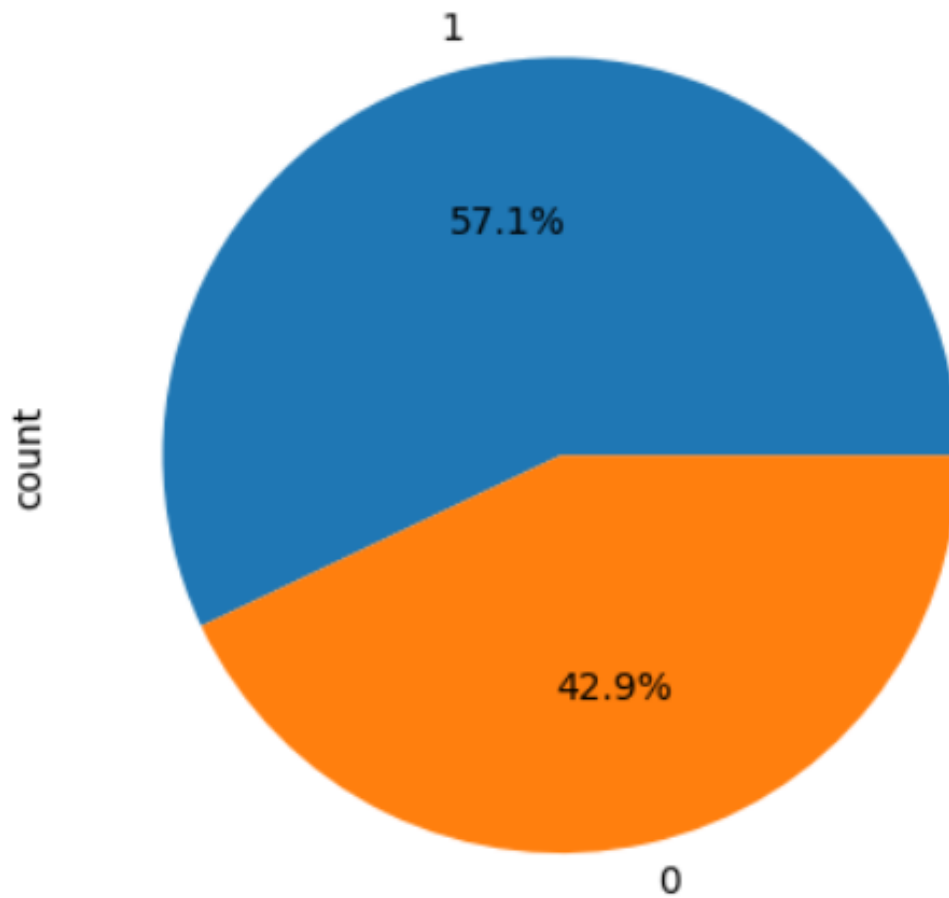


FIGURE 2.5 – Répartition des candidats selon l'expérience de recherche

57.1% des candidats ont une expérience de recherche contre 42.9% qui n'en ont pas. Cela souligne que l'expérience de recherche est un critère courant parmi les postulants, possiblement lié à de meilleures chances d'admission.

Chance of Admit

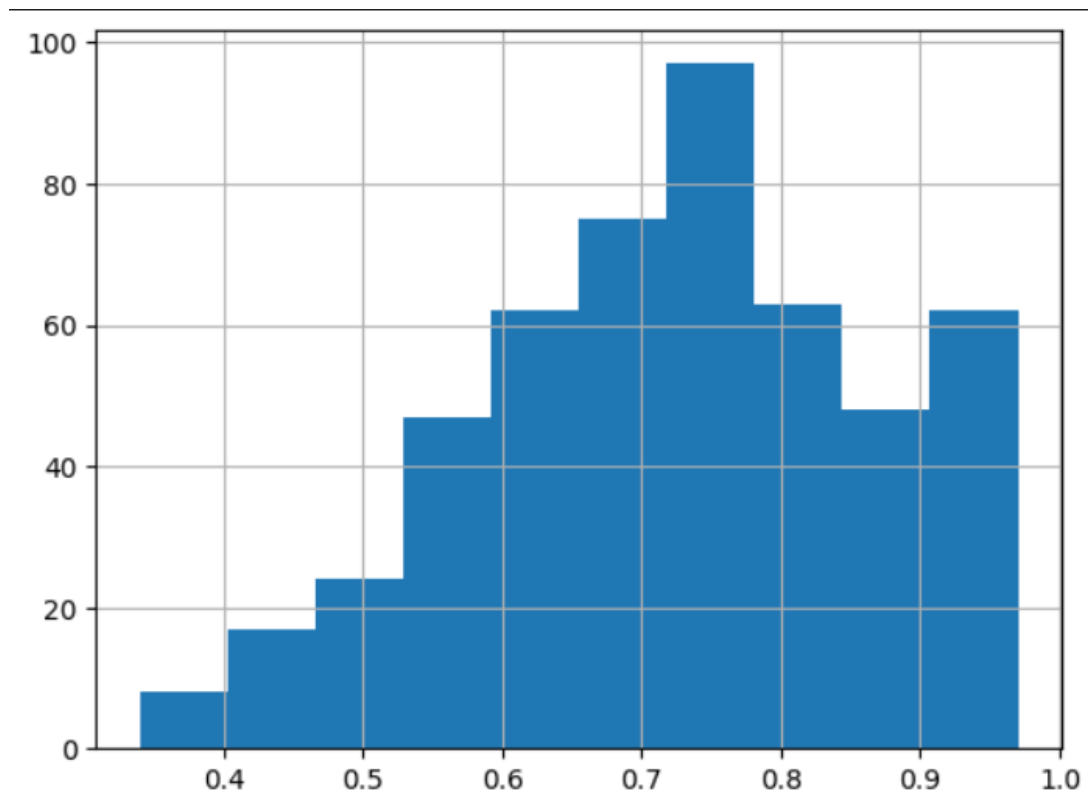


FIGURE 2.6 – Distribution de la probabilité d'admission

La majorité des valeurs de *Chance of Admit* se situe entre 0.6 et 0.8, indiquant une tendance à des profils relativement compétitifs. Une deuxième concentration apparaît entre 0.8 et 0.9, ce qui montre qu'une part non négligeable des candidats a un fort potentiel d'admission.

2.3 Analyse bivariable

GRE Score vs Chance of Admit

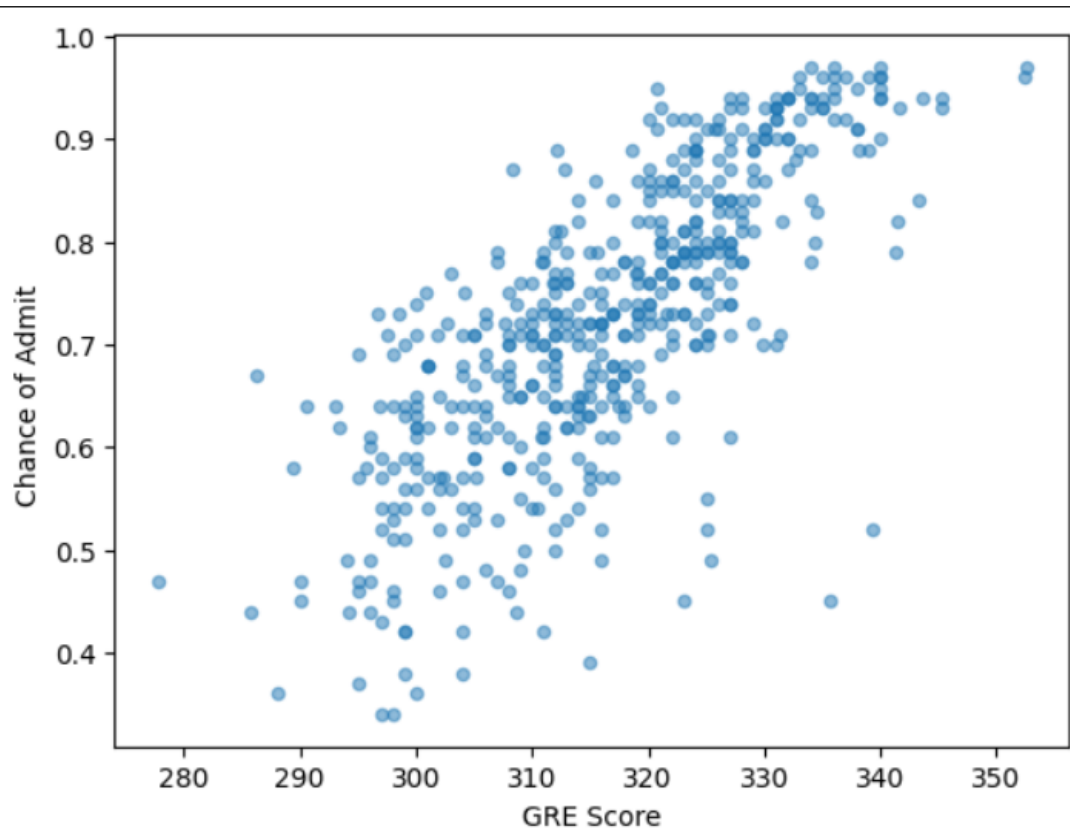


FIGURE 2.7 – Relation entre GRE Score et Chance of Admit

La relation est globalement croissante : les candidats ayant un GRE supérieur à 320 ont davantage de chances d'admission. La concentration est maximale pour des GRE compris entre 300 et 330 et des probabilités d'admission entre 0.6 et 0.8.

CGPA vs Chance of Admit

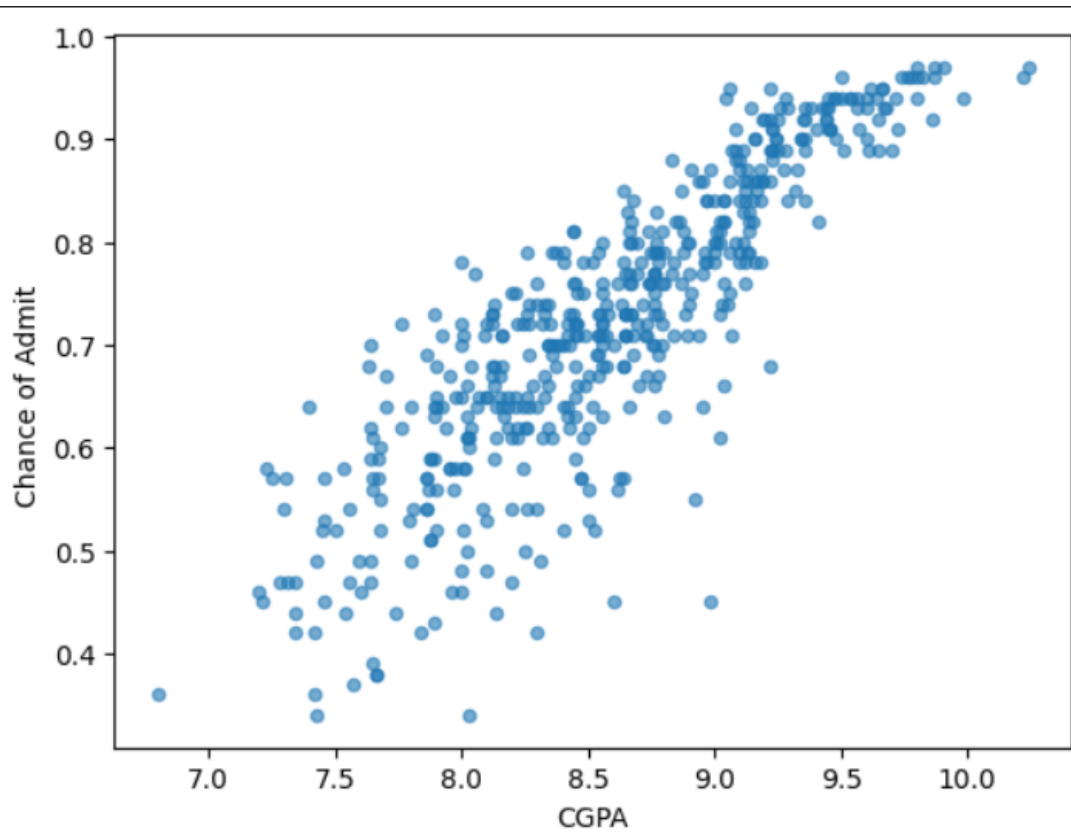


FIGURE 2.8 – Relation entre CGPA et Chance of Admit

Une forte corrélation est visible : plus le CGPA est élevé, plus la probabilité d'admission augmente. L'essentiel des données est concentré entre un CGPA de 8 à 9 et un taux d'admission de 0.6 à 0.8.

TOEFL Score vs Chance of Admit

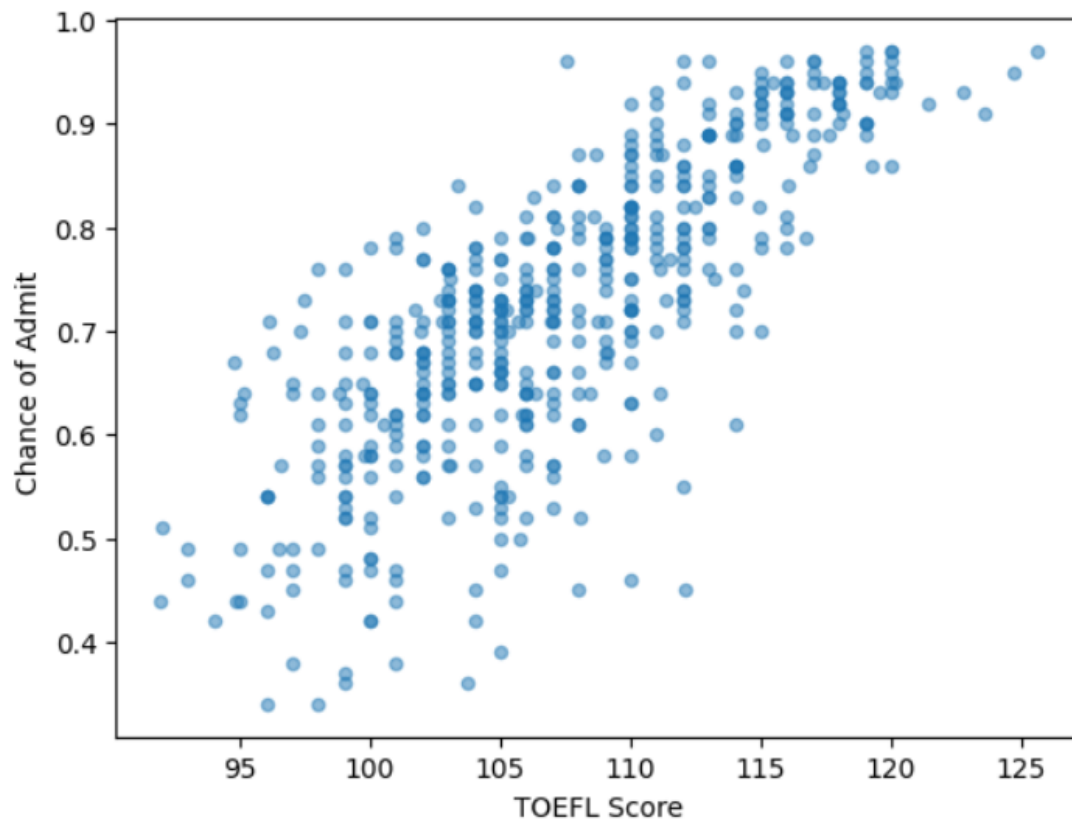


FIGURE 2.9 – Relation entre TOEFL Score et Chance of Admit

La tendance est similaire : les meilleurs scores TOEFL correspondent à des probabilités d'admission plus élevées. La zone la plus dense concerne des scores TOEFL entre 100 et 110, avec des taux d'admission entre 0.6 et 0.8.

Chance of Admit par expérience de recherche

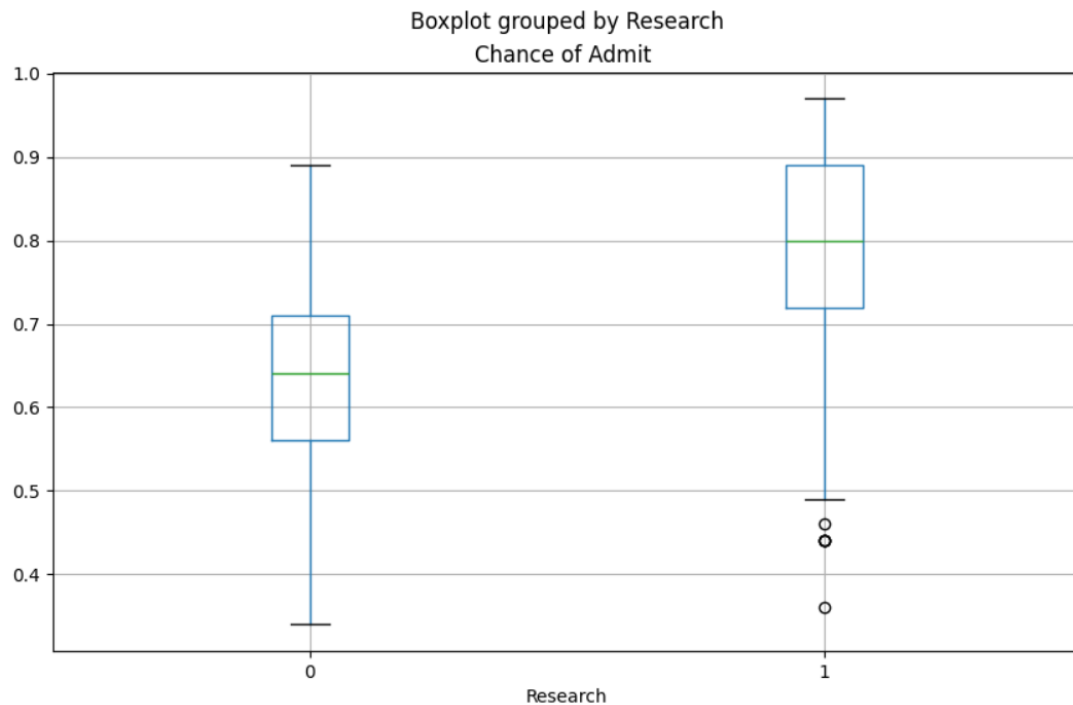


FIGURE 2.10 – Chance of Admit en fonction de l'expérience de recherche

On note une nette différence entre les deux groupes :

- Les candidats sans recherche (0) ont une médiane d'environ 0.65 avec des valeurs centrées entre 0.55 et 0.7.
- Ceux ayant une expérience de recherche (1) ont une médiane proche de 0.8 et un troisième quartile autour de 0.89.

Cela indique que l'expérience de recherche est clairement liée à une meilleure chance d'admission.

Nombre de candidats selon l'expérience de recherche

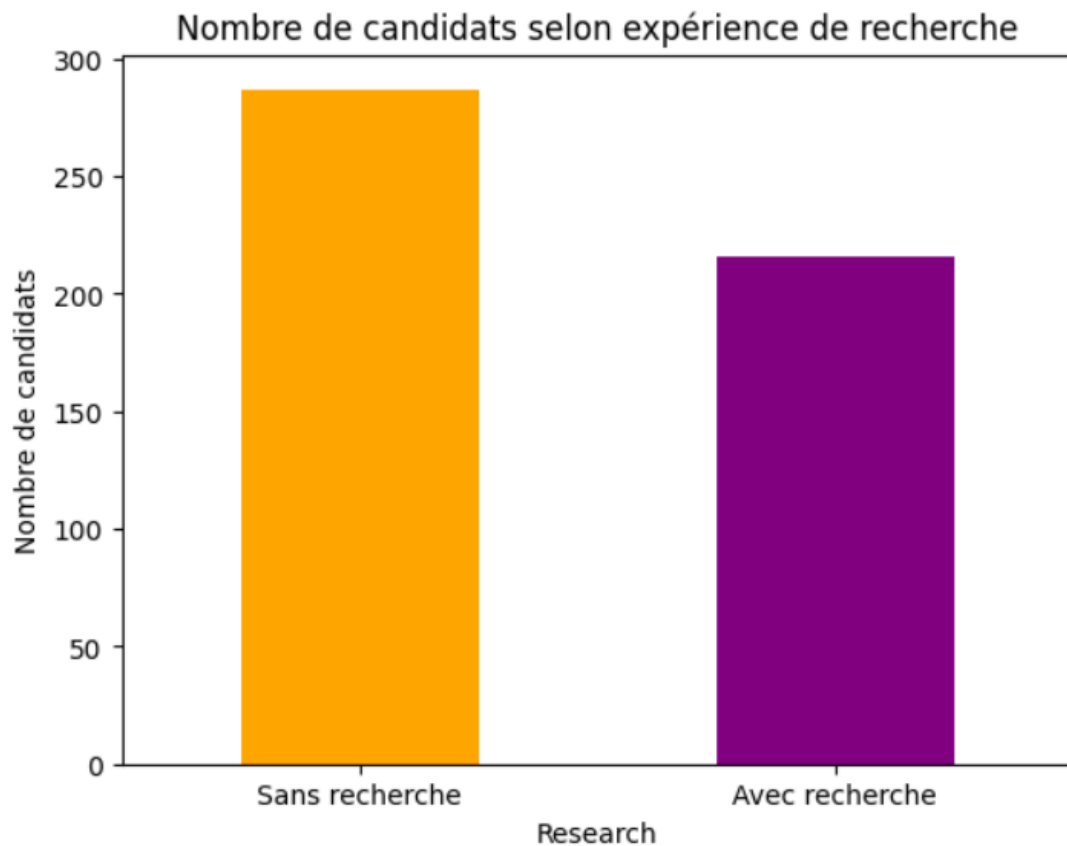


FIGURE 2.11 – Répartition des candidats avec ou sans expérience de recherche

Environ 250 à 300 candidats n'ont pas d'expérience de recherche, tandis que 200 à 250 en ont une. Bien que la majorité ait une expérience, l'écart reste relativement modéré.

2.4 Conclusion

Cette exploration graphique met en évidence plusieurs éléments :

- Une corrélation positive entre le GRE, TOEFL, CGPA et la chance d'admission ;
- L'influence nette de l'expérience de recherche sur la probabilité d'être admis ;
- Une distribution relativement équilibrée des candidats sur plusieurs variables, bien que certaines zones de concentration soient identifiables.

Ces observations seront utilisées pour guider les choix de modélisation dans les prochaines sections.

2.5 Présentation du Dataset

Nom du Dataset :

gc_data.csv

Source :

https://sourceforge.net/projects/triangleinequal/files/Grad%20Cafe%20Data/gc_data.csv/download

Les colonnes qui contiennent dans le dataset :

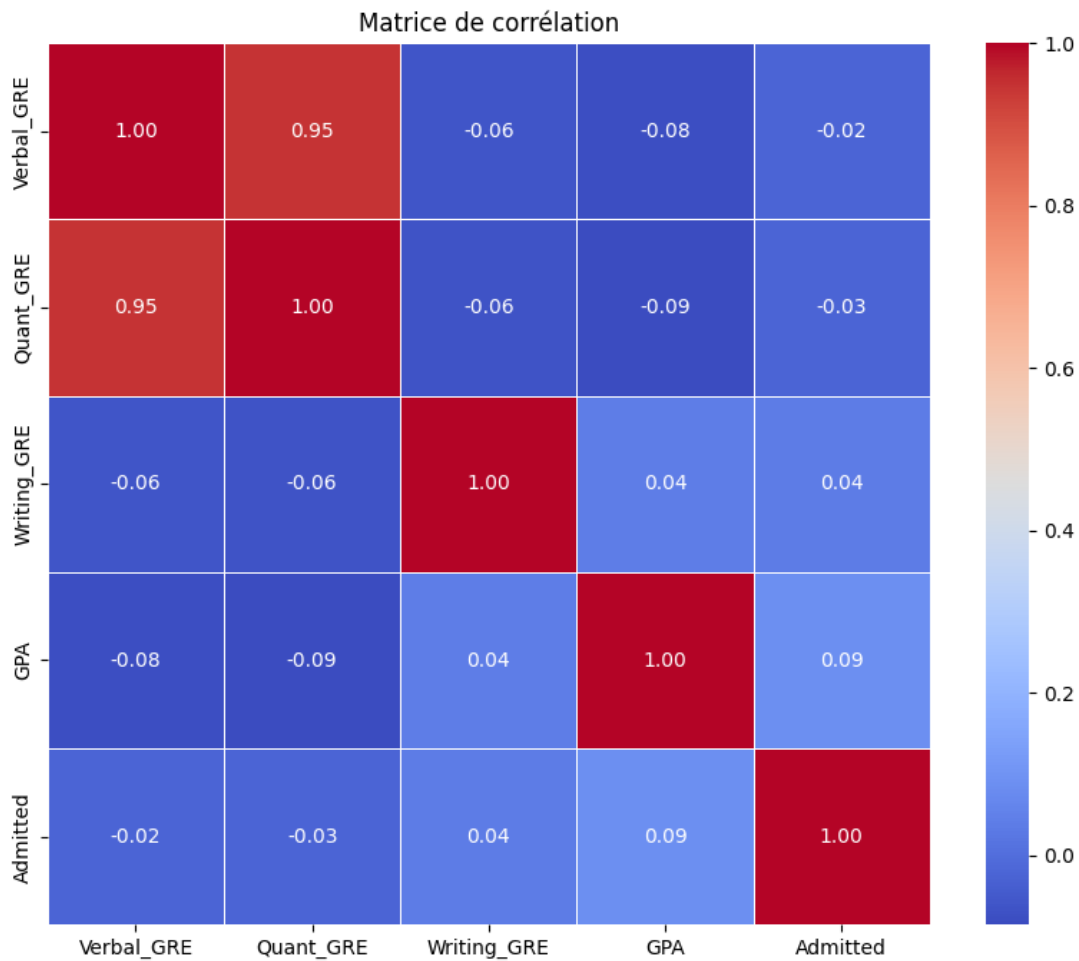
- School, Program, Result, Result_Date, GPA,
- Verbal_GRE, Quant_GRE, Writing_GRE, Subject_GRE,
- Status, Submit_Date, Degree_Type

Aperçu des 5 premières lignes du dataset :

	Verbal_GRE	Quant_GRE	Writing_GRE	GPA	Admitted
1	153.0	168.0	3.0	3.65	1
2	166.0	163.0	4.0	3.89	1
4	162.0	169.0	3.5	3.84	1
6	164.0	164.0	3.5	4.00	1
11	156.0	163.0	4.5	3.60	1

FIGURE 2.12 – Aperçu des 5 premières lignes du dataset

Matrice de Corrélation :



Choix des variables explicatives (X) :

Nous avons sélectionné les variables suivantes comme variables explicatives :

- Verbal_GRE
- Quant_GRE
- Writing_GRE
- GPA
- Result

Variable cible (Y) :

La variable cible est **Admitted**. Cette variable correspond à notre objectif d'étude, car elle représente la classification binaire 1 pour admit et 0 pour le contraire qui sert à prédire si l'étudiant peut être passer ou pas.

Nom du Dataset :

Australian dataset

Source :

<https://www.kaggle.com/datasets/nasirayub2/australian-student-performancedata-aspd24>

Les colonnes qui contiennent dans le dataset :

- 51 colonnes dont :
- GPA
- High School GPA

- Entrance Exam Score
- Attendance
- Family Income
- Parental Education
- Health
- Mental Health
- Exam Scores
- Performance
- ...

Aperçu des 5 premières lignes du dataset :

	Student ID	University ID	University Name	Age	Gender	Major	Year of Study	GPA	High School GPA	Entrance Exam Score	...
0	1	86	University C	25	F	EE	1	2.06	3.50	60	...
1	2	17	University A	26	F	ME	4	2.12	3.46	52	...
2	3	52	University C	20	M	CS	4	2.72	3.33	94	...
3	4	91	University A	25	M	ME	1	3.05	3.54	65	...
4	5	33	University C	22	F	CE	3	1.86	3.35	51	...

FIGURE 2.13 – Aperçu des 5 premières lignes du dataset

Matrice de Corrélation :

Choix des variables explicatives (X) :

Nous avons sélectionné les variables suivantes comme variables explicatives :

- GPA
- Final Exam Scores
- Gender
- Parental Education Level
- Family Income
- Hours of Study per Week

Variable cible (Y) :

La variable cible est **Performance**. Cette variable correspond mais pas à 100% peut être à 70% sur notre objectif d'étude, car elle représente la performance des étudiants mais pas le classifie.

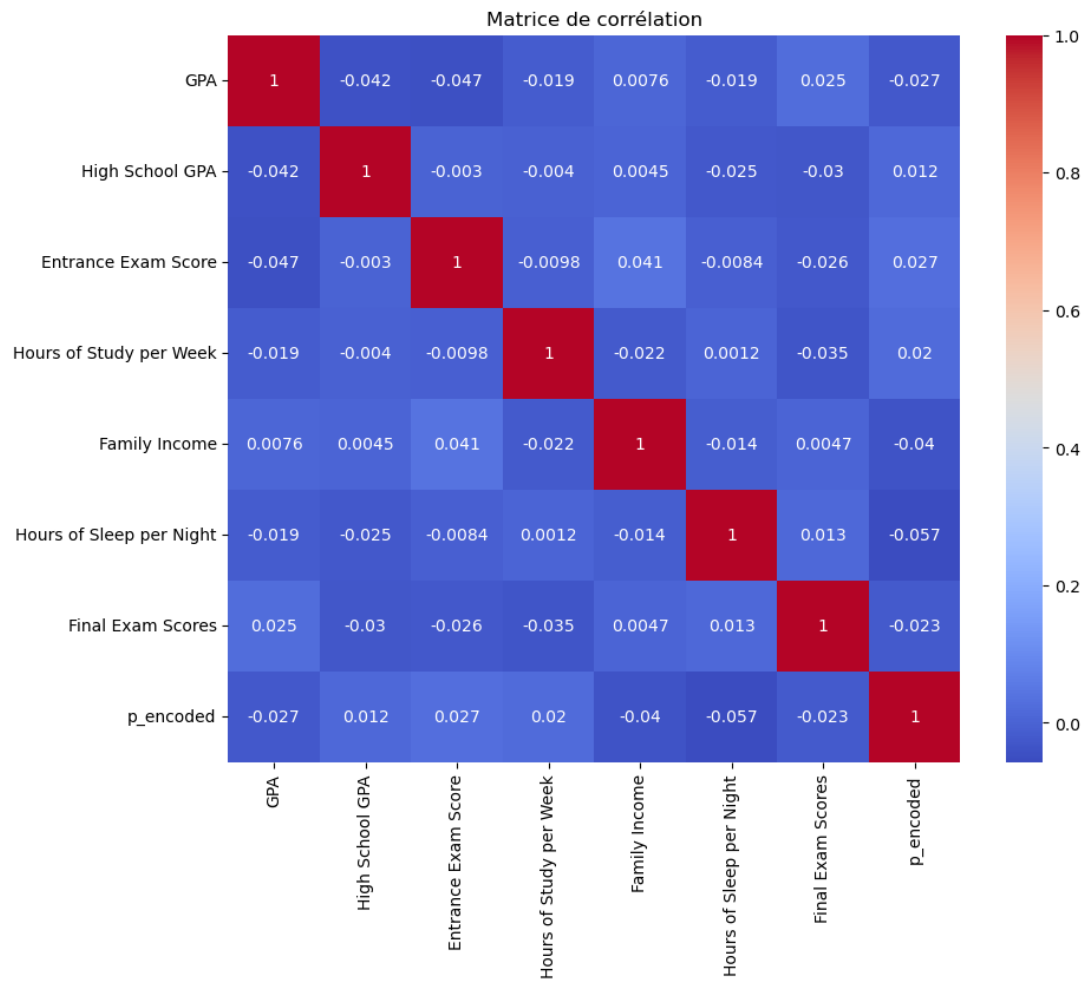


FIGURE 2.14

Chapitre 3

Modélisation

Ici, on a testé différents algorithmes de classification pour prédire les chances pour un étudiants d'entrée dans les écoles aussi dans les universités. On a surtout utilisé : **SVM (SVC)**, **Decision Tree**, **Random Forest**, **Régression logistique** et **K Nearest Neighbours (KNN)**. Pour chaque modèle, on a normalisé les données avec **StandardScaler**.

3.1 Évaluation des modèles

Chaque modèle a été entraîné avec 80% des dataset et testé sur les 20% restants du dataset. Pour comparer leurs performances, on s'est basé sur les métriques suivantes :

- **Accuracy** : le taux de bonnes réponses
- **Precision** : la proportion de vrais positifs parmi les prédictions positives
- **Recall** : la capacité à trouver tous les vrais positifs
- **F1-score** : une moyenne entre la précision et le rappel

3.2 Résultats obtenus

Voici un tableau qui résume les performances de chaque modèle :

Modèle	Accuracy	Précision	Rappel	F1-score
SVC	0.90	0.94	0.91	0.92
Arbre de décision	0.76	0.82	0.82	0.82
Forêt aléatoire	0.87	0.89	0.93	0.91
Régression logistique	0.87	0.92	0.88	0.90
KNN	0.87	0.92	0.88	0.90

TABLE 3.1 – Comparaison des performances des différents modèles

3.3 Courbes d'apprentissage

Pour mieux comprendre le comportement des modèles, on a tracé des courbes d'apprentissage qui montrent comment les performances évoluent avec la quantité de données :

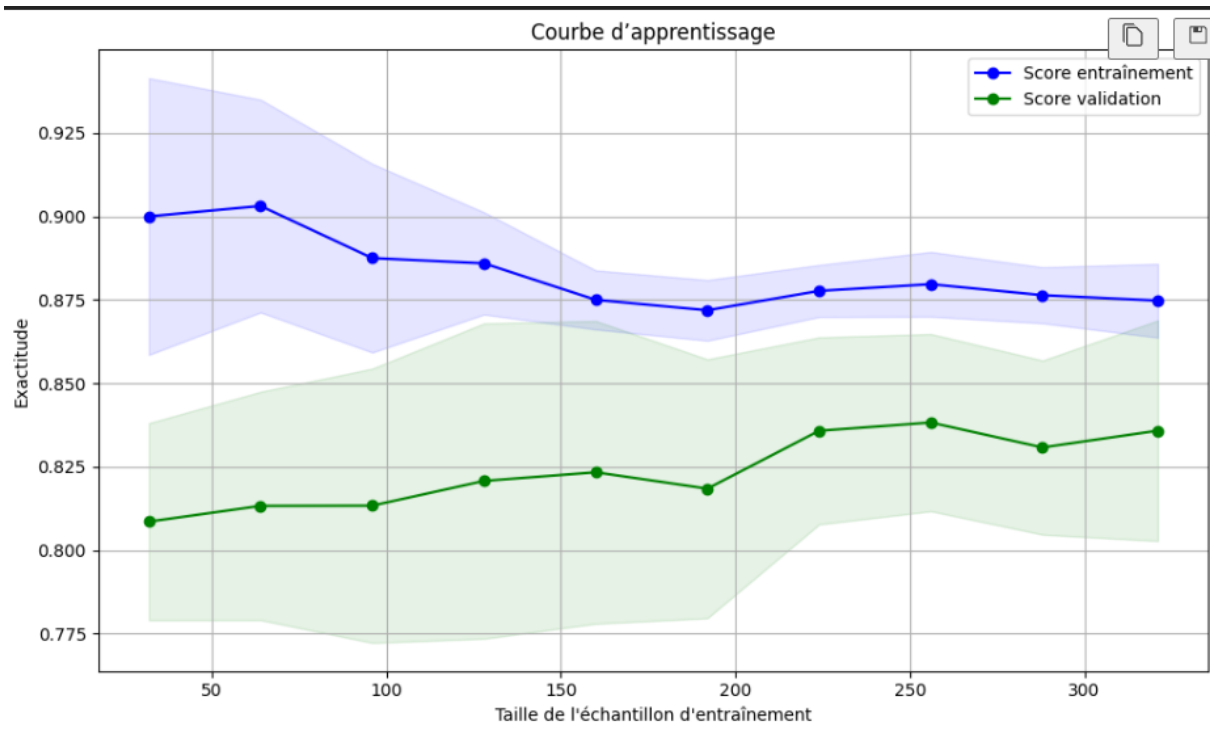


FIGURE 3.1 – Courbe d'apprentissage pour SVC

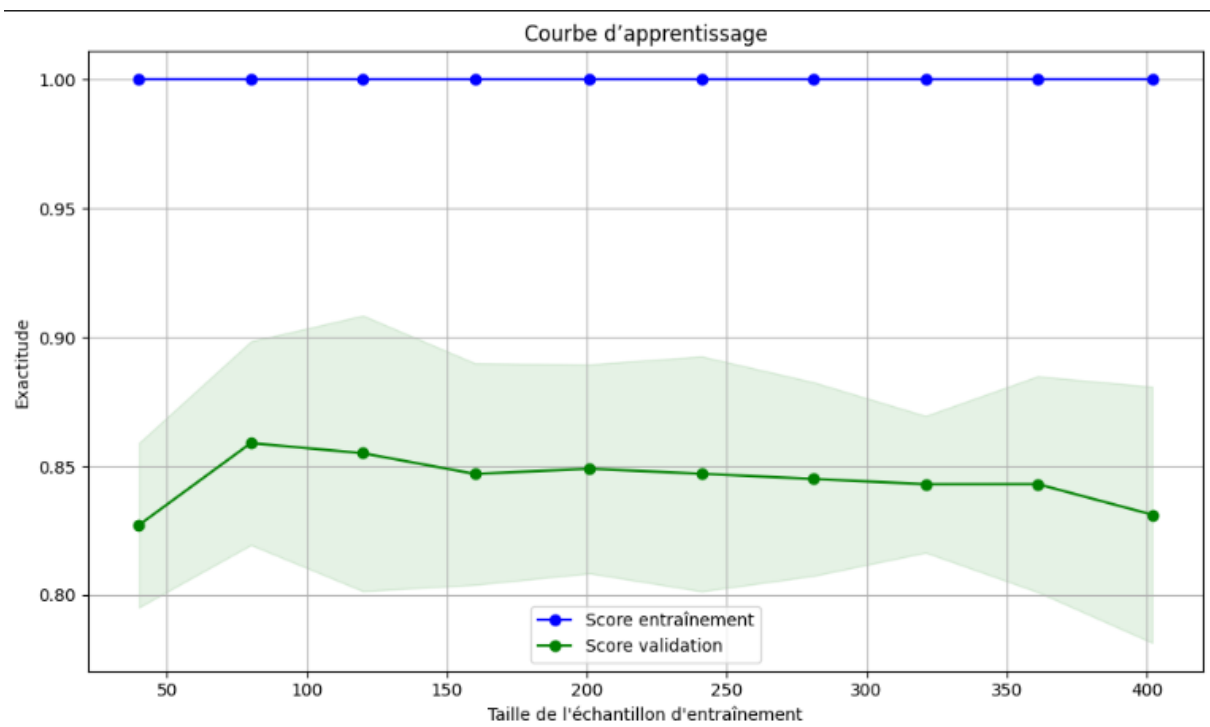


FIGURE 3.2 – Courbe d'apprentissage pour Forêt aléatoire

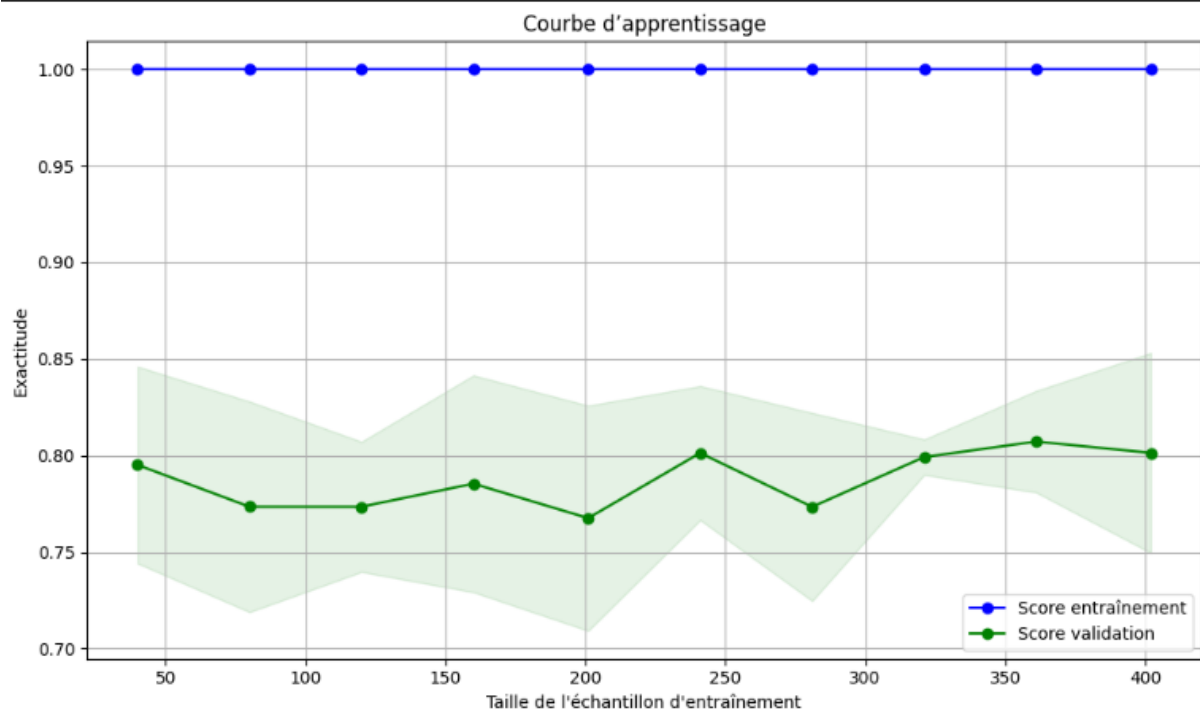


FIGURE 3.3 – Courbe d'apprentissage pour Arbre de décision

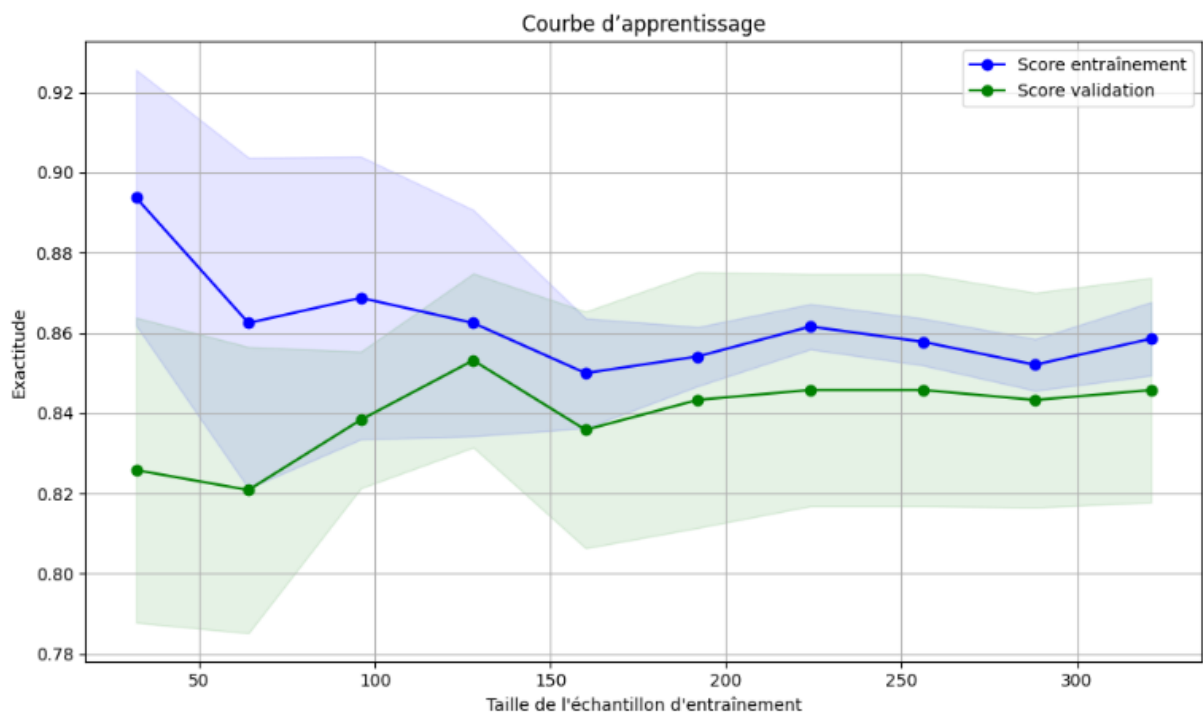


FIGURE 3.4 – Courbe d'apprentissage pour Régression logistique

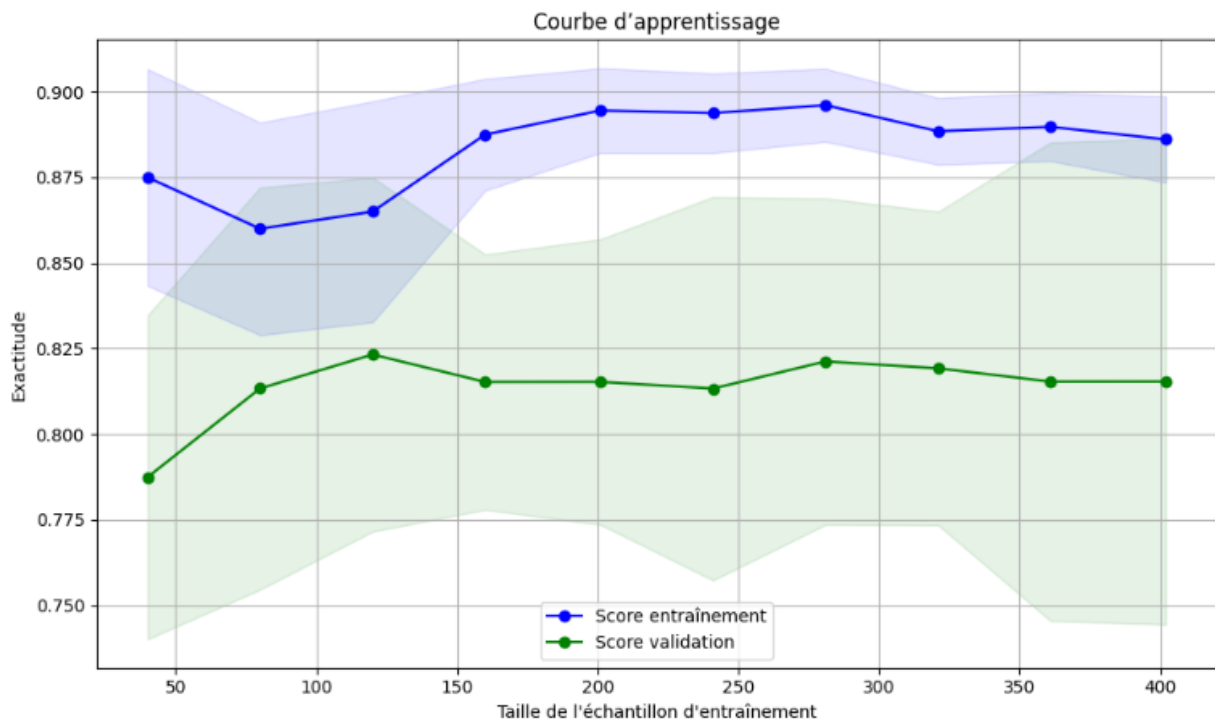


FIGURE 3.5 – Courbe d'apprentissage pour KNN

3.4 Analyse des résultats

En regardant les résultats, on peut faire plusieurs constats intéressants :

- Le modèle **SVC** donne les meilleurs résultats globaux avec un F1-score de 0.92
- Les modèles **Forêt aléatoire** et **KNN** ont des performances assez similaires
- L'**arbre de décision** semble sur-apprendre et a des scores plus bas
- La **régression logistique** se comporte bien et arrive surtout à généraliser : au lieu de juste mémoriser les données, elle apprend vraiment les relations entre les variables

Finalement, même si SVC avait les meilleurs scores, on a choisi la régression logistique pour la suite parce qu'elle est plus stable et plus facile à interpréter. C'est ce modèle qu'on a décidé de sauvegarder pour les prédictions futures.

Chapitre 4

Déploiement

Dans ce chapitre, nous vous présentons comment nous avons rendu notre modèle entraîné accessible et utilisable, en mettant en place une interface simple permettant de tester les prédictions de notre modèle.

4.1 Test interactif du modèle

Afin de permettre une expérience utilisateur intuitive, nous avons développé un script en Python qui invite à saisir pas à pas les informations nécessaires.

Voici comment cela fonctionne :

- L'utilisateur doit renseigner ces notes académiques comme : le **GRE Score**, le **TOEFL Score**, la **Note de l'université**, la force du **SOP** (Statement of Purpose) et du **LOR** (Letter of Recommendation), le **CGPA** (Grade Point Average cumulé), ainsi que son expérience en **recherche**.
- Si une information n'est pas disponible, alors une valeur par défaut sera attribuée, afin de garantir ainsi le bon fonctionnement du modèle même avec des données manquantes.
- Une fois toutes les entrées collectées, le script affiche clairement le résultat sous la forme :
 - `Candidat probablement admis` en cas de prédiction positive.
 - `Candidat probablement non admis` dans le cas contraire.

4.2 Chargement du modèle

Pour pouvoir réutiliser le modèle entraîné sans avoir à le ré-entraîner, nous avons utilisé la bibliothèque `pickle`. Le modèle est donc sauvegardé dans un fichier `logistic.pkl`, ce qui permet de l'utiliser rapidement et de l'intégrer dans n'importe quel site web, etc.

4.3 Exemple d'utilisation

Voici un extrait illustrant la simplicité avec laquelle le modèle peut être chargé et testé :

```
import pickle as pkl
with open('models/logistic.pkl', 'rb') as f:
    model_loaded = pkl.load(f)
test_model_input(model_loaded)
```

Ce prototype nous permet de constituer la première étape essentielle vers un déploiement à plus grande échelle.

Annexes

Bibliothèques utilisées pour l'EDA

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
```

Explication : Utilisation du Bibliothèques pandas pour visualiser les données ,matplotlib et seaborn pour la representation des données a partir des graphes , numpy pour les calculs .

Chargement du dataset

```
df = pd.read_csv('data/Admission_Predict_v2.csv')
```

Explication : Ici on utilise pd.read csv pour pouvoir utiliser le dataset.

Affichage

```
df.head()
```

	Serial No.	GRE Score	TOEFL Score	University Rating	SOP	LOR	CGPA	Research	Chance of Admit	LOR
0	1	337.0	118.0	4	4.5	4.5	9.65	1	0.92	NaN
1	2	324.0	107.0	4	4.0	4.5	8.87	1	0.76	NaN
2	3	316.0	104.0	3	3.0	3.5	8.00	1	0.72	NaN
3	4	322.0	110.0	3	3.5	2.5	8.67	1	0.80	NaN
4	5	314.0	103.0	2	2.0	3.0	8.21	0	0.65	NaN

Explication : En utilisant df.head on affiche les 5 première lignes du dataset.

Aperçu du type des données

```
df.info()
#GRE : Signifie Graduate Record Examination
#TOEFL : Signifie Test of English as a Foreign Language
#SOP : Signifie Statement of Purpose
#utilisé par les comités d'admission pour évaluer les objectifs académiques
#LOR : Signifie Letter of Recommendation
#utilisée pour juger du sérieux, du potentiel et des qualités personnelles du candidat
#CGPA : Signifie Cumulative Grade Point Average
#utilisé pour évaluer la performance académique globale
#Research : Signifie Expérience de recherche cad activité extra-scolaire

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 550 entries, 0 to 549
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Serial No.             550 non-null   int64
1   GRE Score              550 non-null   float64
2   TOEFL Score            513 non-null   float64
3   University Rating      550 non-null   int64
4   SOP                    513 non-null   float64
5   LOR                    550 non-null   float64
6   CGPA                   550 non-null   float64
7   Research                550 non-null   int64
8   Chance of Admit        550 non-null   float64
9   LOR                    0 non-null     float64
dtypes: float64(7), int64(3)
memory usage: 43.1 KB
```

Explication : Avec `df.info` on a une aperçu globale du chaque type de données.

Resumer du dataset

```
df.describe()
```

	Serial No.	GRE Score	TOEFL Score	University Rating	SOP	LOR	CGPA	Research	Chance of Admit	LOR
count	550.000000	550.000000	513.000000	550.000000	513.000000	550.000000	550.000000	550.000000	550.000000	0.0
mean	250.896364	316.335651	107.204041	3.125455	3.376218	3.484545	8.591772	0.572727	0.721655	NaN
std	146.927197	12.674963	6.434952	1.135889	0.995978	0.925375	0.616908	0.495133	0.142310	NaN
min	1.000000	277.754368	91.916154	1.000000	1.000000	1.000000	6.800000	0.000000	0.340000	NaN
25%	121.250000	308.000000	102.725335	2.000000	2.500000	3.000000	8.131200	0.000000	0.630000	NaN
50%	251.500000	316.000000	107.000000	3.000000	3.500000	3.500000	8.589128	1.000000	0.720000	NaN
75%	378.750000	325.000000	112.000000	4.000000	4.000000	4.000000	9.060000	1.000000	0.830000	NaN
max	500.000000	353.144823	125.588646	5.000000	5.000000	5.000000	10.240747	1.000000	0.970000	NaN

Explication : `df.describe` nous permet rapidement les statistiques des colonnes numériques.

Suppression des colonnes inutiles

```
df_clean = df.drop(columns=[df.columns[9] , df.columns[0]])
```

Explication : On supprime la colonne Serial No. qui est inutile durant l'entraînement de notre model et on supprime aussi le doublon de la colonne 'LOR'.

Nettoyage des données

```
df_clean.isna().sum()
```

GRE Score	0
TOEFL Score	37
University Rating	0
SOP	37
LOR	0
CGPA	0
Research	0
Chance of Admit	0
dtype:	int64

```
df_clean = df_clean.dropna()
```

Explication : On compte le nombre de valeur manquantes grace à `df_clean.isna().sum()`, après avoir fait ça on fait `df_clean.dropna()` pour supprimer ces valeurs manquantes.

Visualisation univariée des données

```
df_clean["GRE Score"].hist()
```

```
df_clean["CGPA"].hist()
```

```
df_clean["TOEFL Score"].hist()
```

```
df_clean["University Rating"].hist()
```

```
df_clean["Chance of Admit "].hist()
```

```
df_clean["Research"].value_counts().plot(kind='pie', autopct='%1.1f%%')
```

Visualisation bivariée des données

```
df_clean.plot.scatter(x='GRE Score', y='Chance of Admit ', colormap='viridis', alpha=0.5)
```

```
df_clean["GRE Score"].corr(df_clean["Chance of Admit "])
```

```
df_clean.plot.scatter(x='CGPA', y='Chance of Admit ', colormap='viridis', alpha=0.6)
```

```
df_clean["CGPA"].corr(df_clean["Chance of Admit "])
```

```
df_clean.plot.scatter(x='TOEFL Score', y='Chance of Admit ', colormap='viridis', alpha=0.5)
```

```
df_clean["TOEFL Score"].corr(df_clean["Chance of Admit "])
```

```
df_clean.boxplot(column='Chance of Admit ', by='Research', figsize=(10, 6))
```

```
df_clean["Research"].corr(df_clean["Chance of Admit "])
```

```
corr_matrix = df_clean.corr(numeric_only=True)
corr_matrix
```

```
# Visualiser la corrélation entre la moyenne générale (CGPA) et la chance d'admission.
plt.scatter(df_clean['CGPA'], df_clean['Admitted'], alpha=0.6, color='green')
plt.title("CGPA vs Admission")
plt.xlabel("CGPA")
plt.ylabel("Admit")
plt.show()
```

```
sns.boxplot(x='University Rating', y='TOEFL Score', data=df_clean)
plt.title("Distribution TOEFL selon University Rating")
plt.show()
```

```
sns.pairplot(df_clean[['GRE Score', 'TOEFL Score', 'CGPA', 'Admitted']].dropna())
plt.suptitle("Relations entre variables clés", y=1.02)
plt.show()
```

Variable utilisée pour l'entraînement

```
● #Les colonnes qu'on utilisera pour entrainer le modèle
features = ['GRE Score', 'TOEFL Score', 'University Rating', 'SOP', 'LOR ', 'CGPA', 'Research']
x = df_clean[features]
y = df_clean['Admitted']
```

Bibliothèques utilisées pour l'entraînement du modèle

```
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, learning_curve, cross_val_score
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.pipeline import Pipeline
```

Exemple de code utilisé pour l'entraînement du modèle

```

# Pipeline
rfc_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', RandomForestClassifier(random_state=42))
])

# Courbe d'apprentissage
train_sizes, train_scores, val_scores = learning_curve(
    rfc_pipeline,
    X, y,
    train_sizes=np.linspace(0.1, 1.0, 10),
    cv=5,
    scoring='accuracy',
    shuffle=True,
    random_state=42
)

# Moyennes et écarts-types
train_mean = np.mean(train_scores, axis=1)
val_mean = np.mean(val_scores, axis=1)
train_std = np.std(train_scores, axis=1)
val_std = np.std(val_scores, axis=1)

# Affichage 1 : simple
plt.figure(figsize=(10, 5))
plt.plot(train_sizes, train_mean, 'o-', Label="Train Accuracy")
plt.plot(train_sizes, val_mean, 'o-', Label="Validation Accuracy")
plt.title("Learning Curve")
plt.xlabel("Nombre d'échantillons d'entraînement")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.show()

# Affichage 2 : avec bandes d'incertitude
plt.figure(figsize=(10, 6))
plt.plot(train_sizes, train_mean, 'o-', color='blue', Label='Score entraînement')
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, alpha=0.1, color='blue')

plt.plot(train_sizes, val_mean, 'o-', color='green', Label='Score validation')
plt.fill_between(train_sizes, val_mean - val_std, val_mean + val_std, alpha=0.1, color='green')

plt.title('Courbe d'apprentissage')
plt.xlabel('Taille de l\'échantillon d\'entraînement')
plt.ylabel('Exactitude')
plt.legend(loc='best')
plt.grid(True)
plt.tight_layout()
plt.show()

```


Exemple de code utilisé pour tester le modèle

```
def test_model_input(model):
    print("=== Prédiction d'admission à l'université ===")
    print("Tape 'aucun' si tu n'as jamais passé un test ou fourni un document.\n")

    # GRE Score
    while True:
        gre_input = input("GRE Score (260-340 ou 'aucun') : ").strip().lower()
        if gre_input == 'aucun':
            gre = 300 # valeur par défaut neutre
            print("GRE non fourni. Valeur par défaut utilisée : 300")
            break
        try:
            gre = float(gre_input)
            if 260 <= gre <= 340:
                break
            else:
                print("Valeur hors plage.")
        except ValueError:
            print("Entrée invalide.")

    # TOEFL Score
    while True:
        toefl_input = input("TOEFL Score (0-120 ou 'aucun') : ").strip().lower()
        if toefl_input == 'aucun':
            toefl = 90 # valeur par défaut neutre
            print("TOEFL non fourni. Valeur par défaut utilisée : 90")
            break
        try:
            toefl = float(toefl_input)
            if 0 < toefl <= 120:
                break
            else:
                print("Valeur hors plage.")
        except ValueError:
            print("Entrée invalide.")

    # University Rating
    while True:
        ur_input = input("Note de l'université (1 à 5 ou 'aucun') : ").strip().lower()
        if ur_input == 'aucun':
            ur = 3 # neutre
            print("Note d'université non fournie. Valeur par défaut utilisée : 3")
            break
        try:
            ur = int(ur_input)
            if 1 <= ur <= 5:
                break
            else:
                print("Valeur hors plage.")
        except ValueError:
            print("Entrée invalide.")

    # SOP
    while True:
        sop_input = input("SOP (Lettre de motivation notée de 0.0 à 5.0 ou 'aucun') : ").strip().lower()
        if sop_input == 'aucun':
```

```
    # GRE Score
    while True:
        gre_input = input("GRE Score (260-340 ou 'aucun') : ").strip().lower()
        if gre_input == 'aucun':
            gre = 300 # valeur par défaut neutre
            print("GRE non fourni. Valeur par défaut utilisée : 300")
            break
        try:
            gre = float(gre_input)
            if 260 <= gre <= 340:
                break
            else:
                print("Valeur hors plage.")
        except ValueError:
            print("Entrée invalide.")

    # TOEFL Score
    while True:
        toefl_input = input("TOEFL Score (0-120 ou 'aucun') : ").strip().lower()
        if toefl_input == 'aucun':
            toefl = 90 # valeur par défaut neutre
            print("TOEFL non fourni. Valeur par défaut utilisée : 90")
            break
        try:
            toefl = float(toefl_input)
            if 0 < toefl <= 120:
                break
            else:
                print("Valeur hors plage.")
```

bibliographie

Dans cette section, nous listons l'ensemble des sources consultées pour la réalisation de ce travail, principalement des sites web fournissant des données, articles ou documentations pertinentes.

1. Dataset principale. *admission_rate_vr1.csv*. Disponible sur : . Consulté le 25 août 2025.

Dataset test. *gc_data.csv*. Disponible sur : . Consulté le 20 août 2025.

Dataset test. *Predict students dropout and academic success*. Disponible sur : <https://archive.ics.uci.edu/dataset/697/predict+students+dropout+and+academic+success>. Consulté le 18 août 2025.

Dataset test. *Graduate school admission datab*. Disponible sur : https://figshare.com/articles/dataset/Graduate_school_admission_data/19561582. Consulté le 22 août 2025.

TheGradCafe. *How to scrape the website of thegradcafe*. Disponible sur : forum.thegradcafe.com/topic/51483-scraped-admissions-data. Consulté le 21 août 2025.

Chapitre 5

Conclusion

Ce travail de recherche nous a permis d'aboutir à la création et à la validation d'un modèle prédictif dont l'objectif principale est d'évaluer les chances d'admission d'un étudiant, en s'appuyant sur le dataset La methodology qu'on a adoptée repose sur ces successions d'étapes clés comme :

- **Préparation des données** : Cette phase préliminaire, bien que souvent sous-estimée, s'est avérée déterminante. Elle a inclus le nettoyage des données, leur exploration approfondie, et la sélection des variables principales pour la prédiction.
- **Modélisation** : Une comparaison approfondie parmi plusieurs algorithmes d'apprentissage supervisé a été réalisée. Nous avons confronté leurs performances comme les performances du SVC, random forest, l'Arbre de Décision, la Régression Logistique et les K Nearest Neighbors, cherchant ainsi à trouver lequel de ces algorithmes est le plus précis pour ainsi pouvoir le prendre.
- **Évaluation** : Au-delà des métriques classiques de performance comme l'exactitude, la précision ou le score F1, nous avons analysé les courbes d'apprentissage pour détecter et comprendre les écueils classiques que sont le surajustement ou le sous-ajustement, garantissant ainsi la robustesse de notre modèle.
- **Déploiement** : Pour concrétiser cette étude, une application interactive a été développée. Elle permet à un utilisateur de saisir ses informations et d'obtenir une estimation personnalisée, matérialisant ainsi l'utilité pratique du modèle.

L'examen méticuleux des performances a révélé que la **Régression Logistique** s'est imposée comme l'approche la plus pertinente pour notre jeu de données. Son excellence ne réside pas dans une complexité algorithmique supérieure, mais plutôt dans son équilibre et sa remarquable capacité à généraliser, évitant ainsi les pièges du surajustement dans lesquels sont tombés certains de ses concurrents. Elle a su capturer avec finesse les relations fondamentales liant les variables explicatives au résultat à prédire.

Enfin, ce mémoire ne se contente pas de présenter un modèle opérationnel ; il ouvre la voie à de multiples prolongements. Le développement d'une interface web intuitive ou la publication sous forme d'API constituent des évolutions naturelles pour étendre la portée de cet outil.

Par ailleurs, cette recherche vient rappeler, s'il le fallait encore, que la performance d'un modèle de machine learning ne naît pas de la sophistication de son algorithme, mais bien de la qualité du travail en amont : la préparation des données et leur analyse critique restent la pierre angulaire de tout projet réussi en science des données.